

Experiments Summary

The variable factors that affect the training process:

Data:

- Size
 - We worked usually on two sizes:
 - Small Dataset: 10% of the Dataset
 - Medium Dataset : 50% of the dataset
- Construction: 1D or 2D array
 - in our case we used 2D arrays
- Representation : binary and hex
- Padding: Center or right
 - Center: The padding element (0) is added on both sides of the example 000101000
 - Right : The padding element is added only to the right side 101000000

Loss Function:

- Cross Entropy
 - Wasn't improving much
- L1
 - Had an increasing loss
- Binary cross entropy (most used)
 - Achieved best results

Optimizer:

- Adam: torch default
 - betas = (0.9, 0.999)
 - Learning rate of 1e-3 and rescheduled to 1e-4

Activation Function:

- elu
- Sigmoid for binary data representation
- Softmax with hex representation

Architecture:

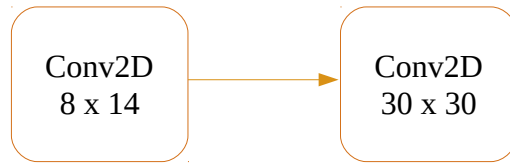
1. Customized
2. Pre-trained
3. Hybrid

Customized models:

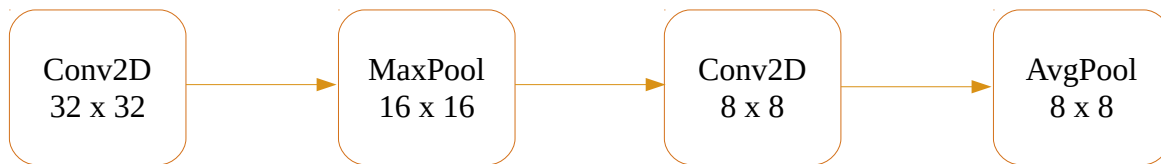
Models always consist of encoder and decoder for simplicity we will mention the encoder only, the decoder will be its reverse with the exact same dimensions.

We **always** use stride of 1 and activation of **elu** except in last layer.

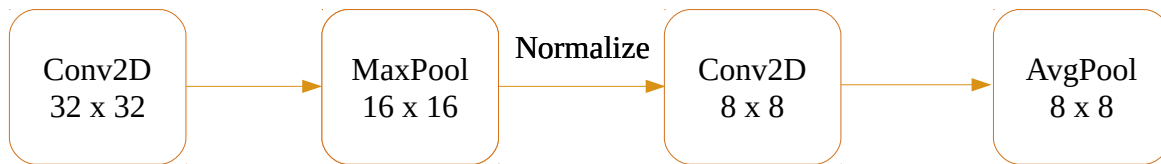
One to one model:



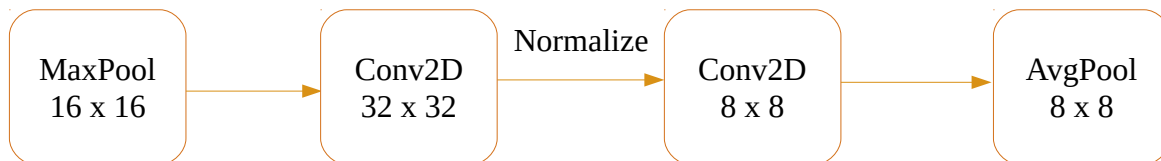
four_layer:



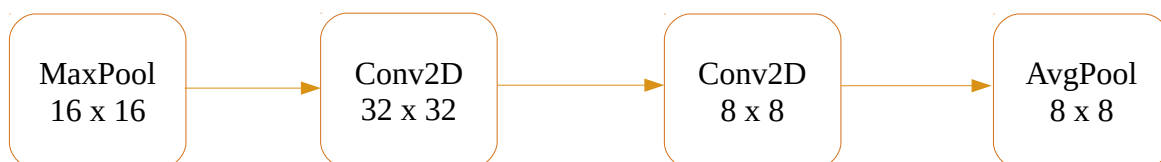
Normalize



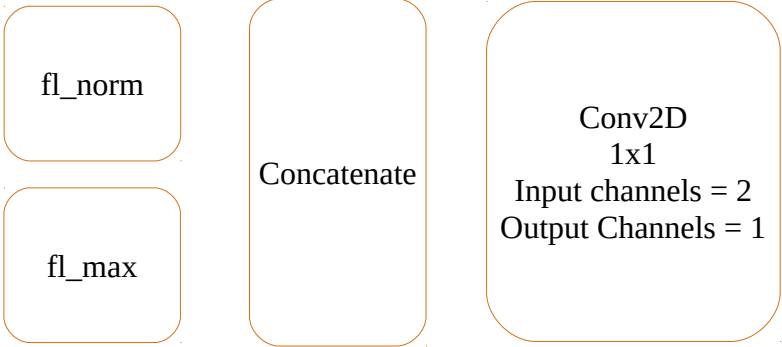
fl_max_norm: (four layer max pooling first with normalization)



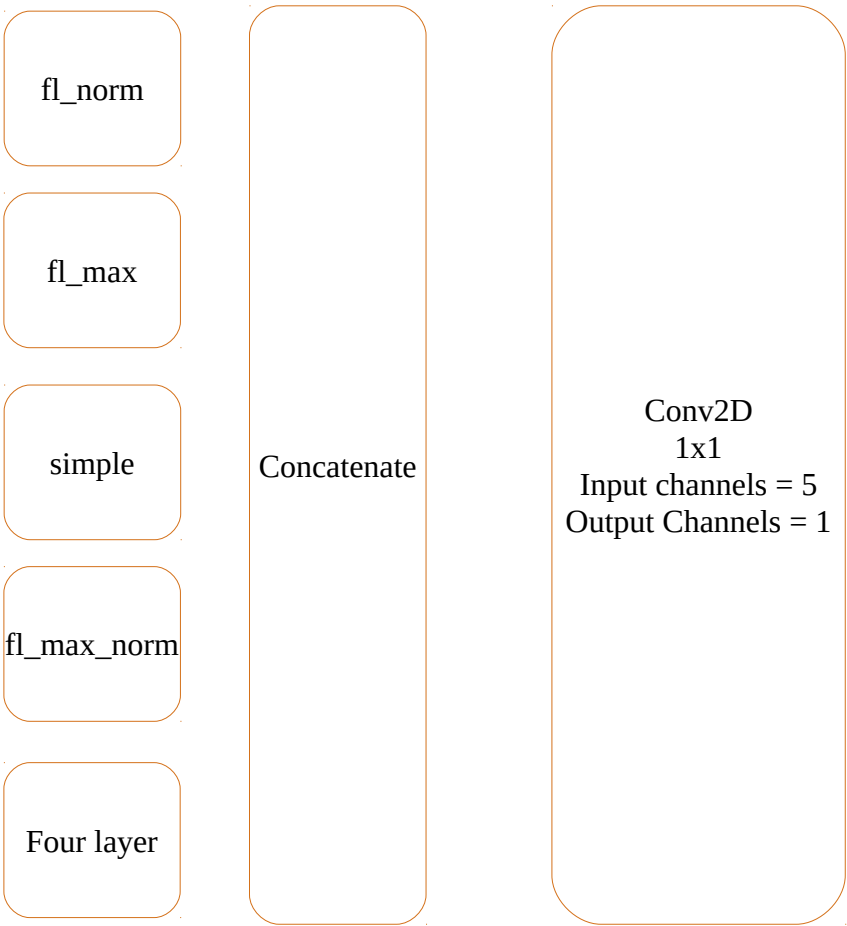
fl_max: (four layer max pooling first without normalization)



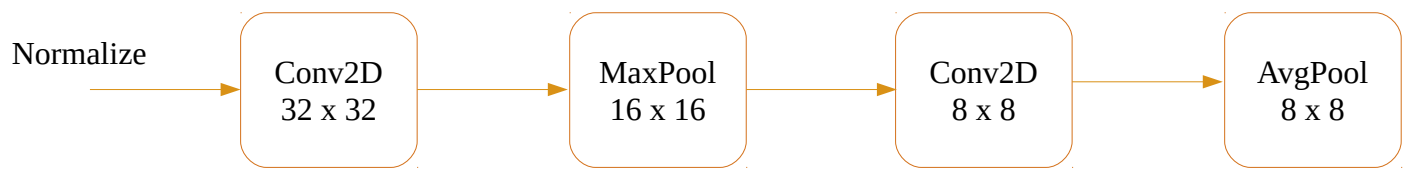
Combined



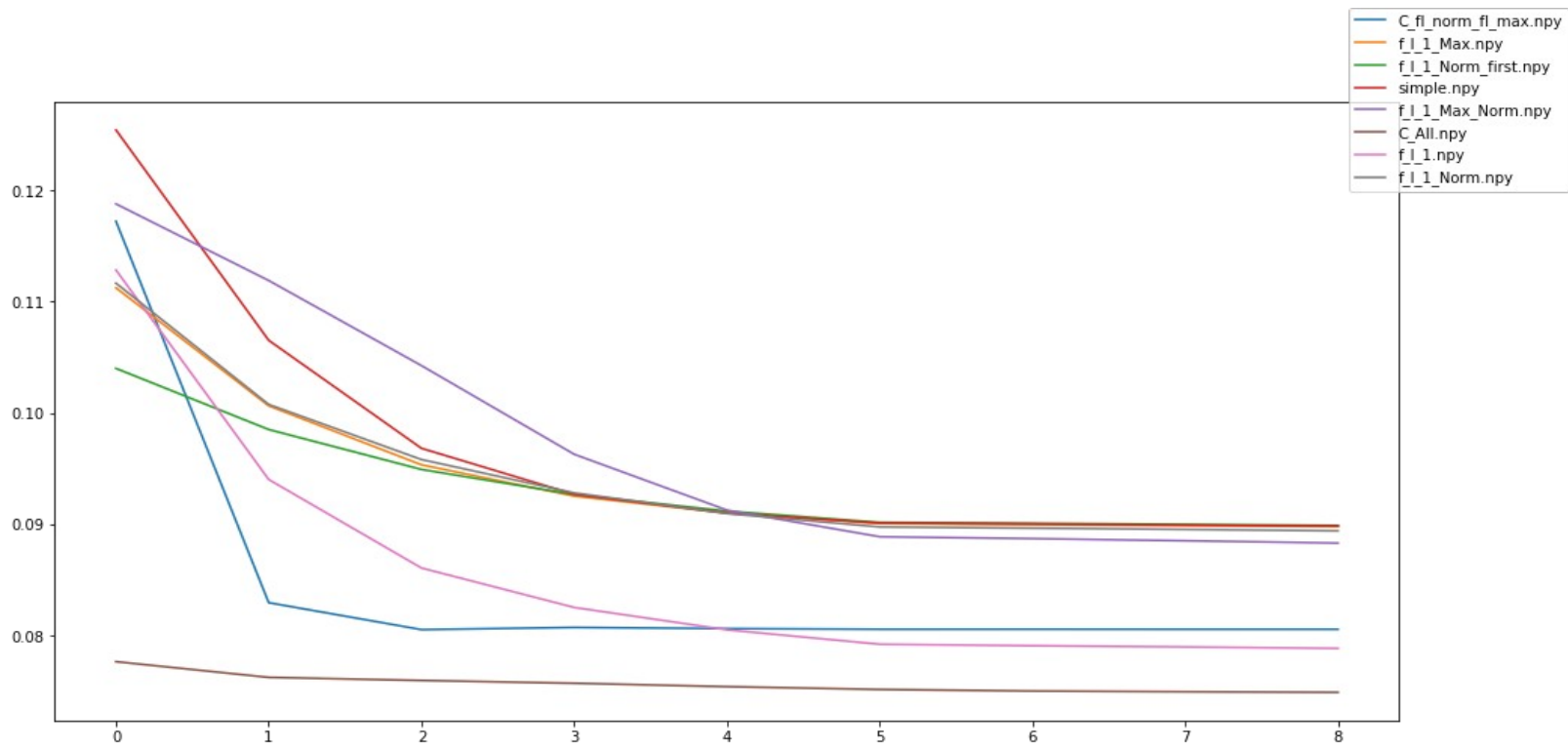
All in All



f_l_1_Norm_first: (Normalize input first)



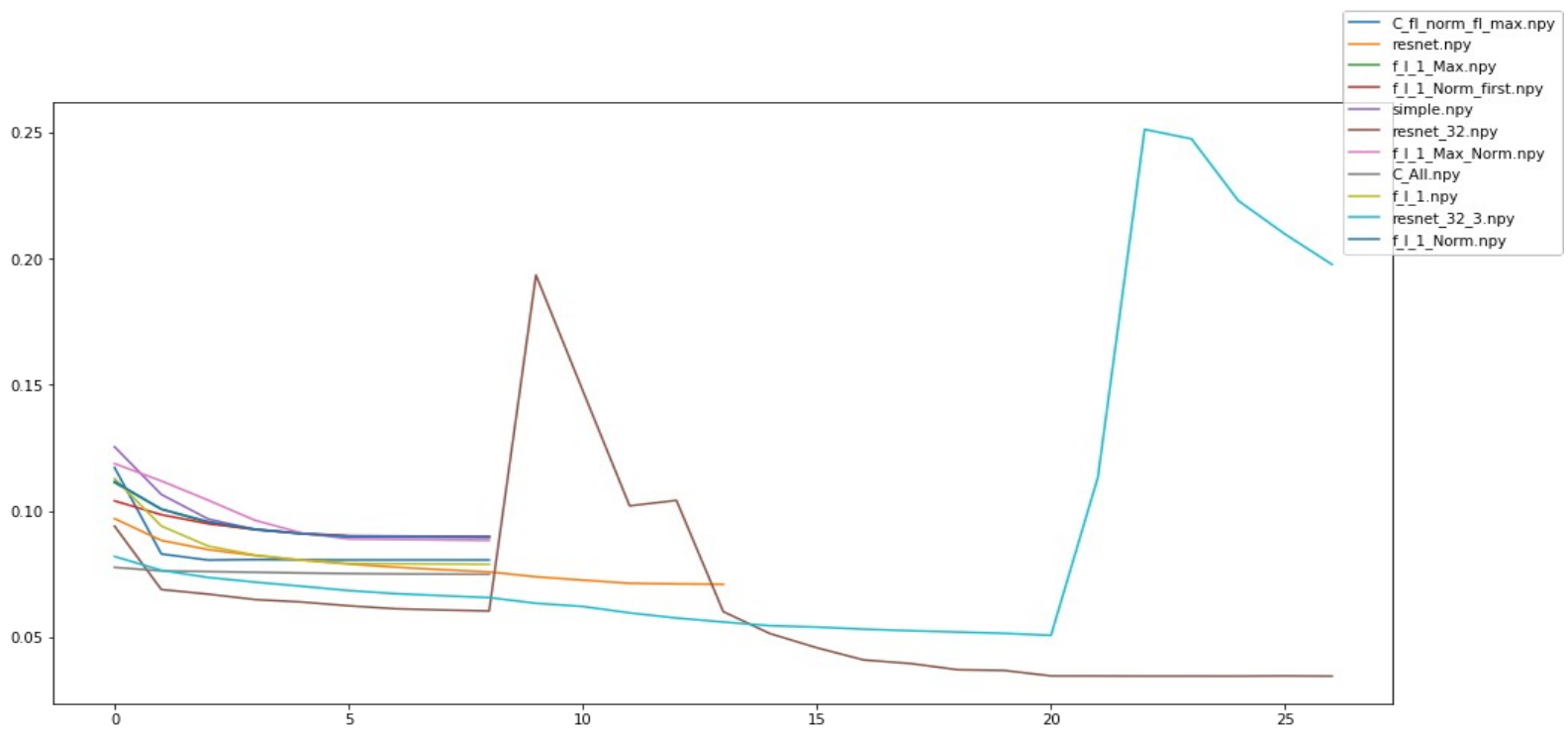
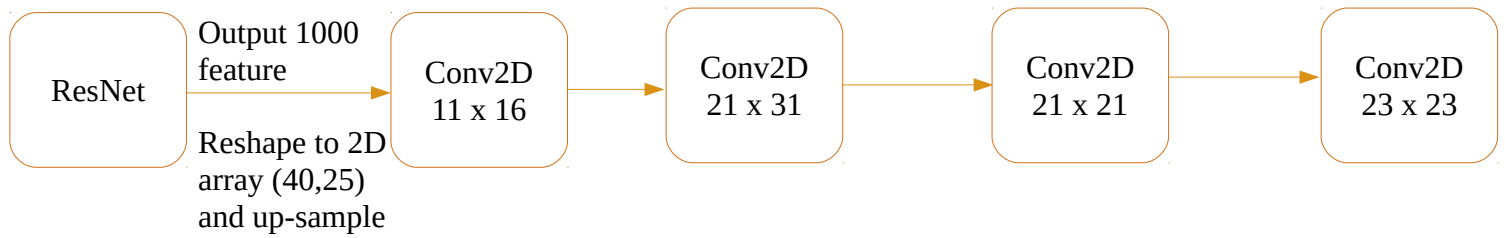
Compare results



Hybrid Models

1. Resnet18
2. U-Net

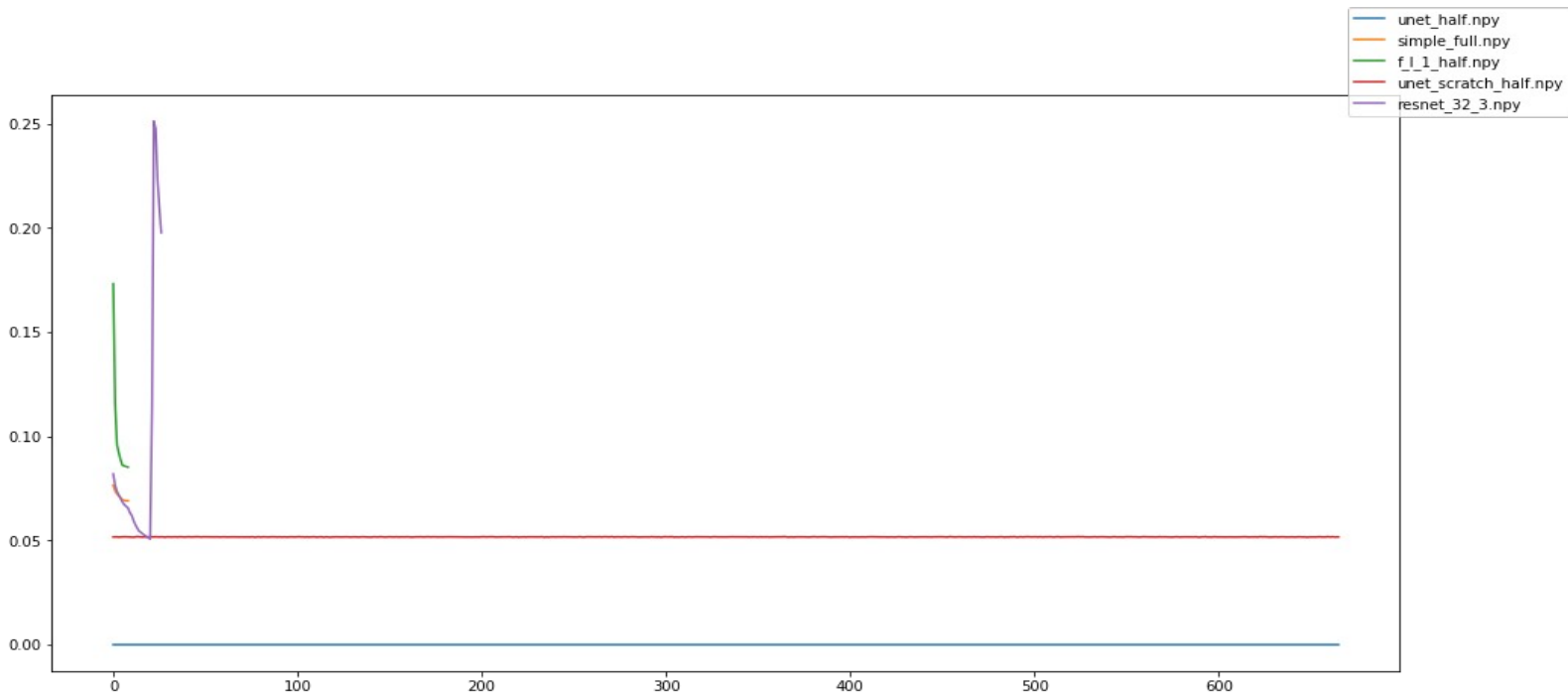
Resnet18



U-Net

U-Net by nature has an internal architecture that looks like an encoder so no extra layers were added

We experimented pre-trained U-net weights and from scratch training

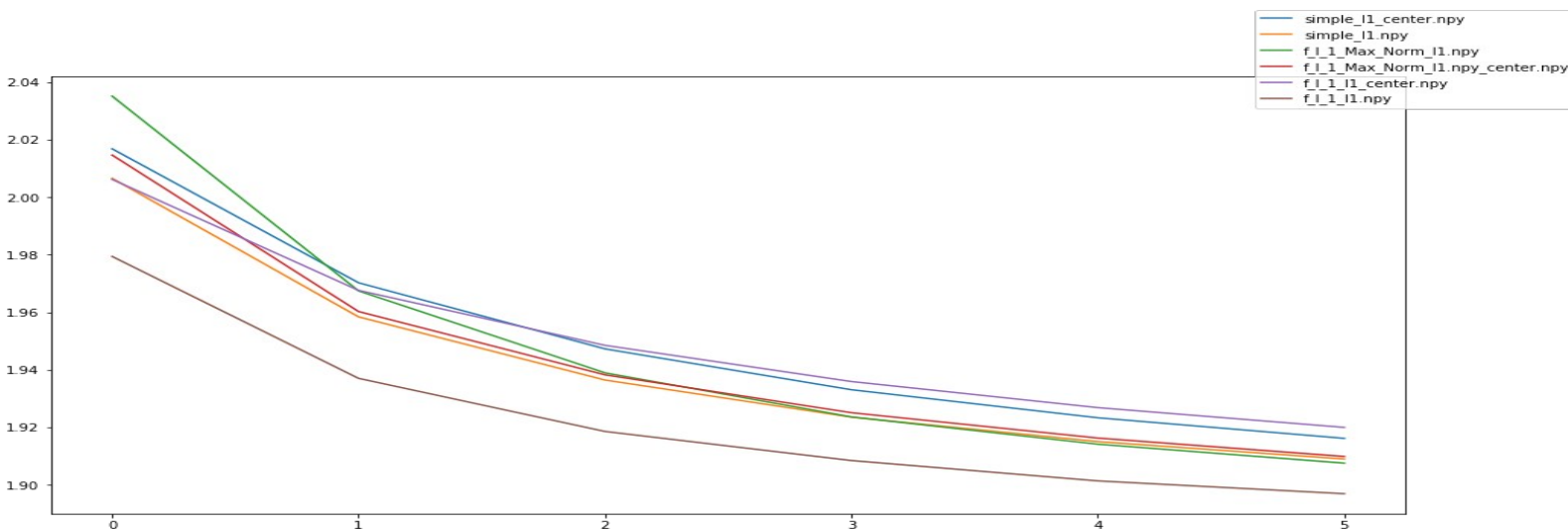


Model based on Hex representation:

Generally its impleneted by making the output of 2d layer to be 16 channel(number of hex) and apply softmax of the number of channel axis.

Since in hex representation number of elements is much less than binary most of the image will be padding so we compare between Center and right padding in the following image.

Model chives slightly better on Right padding

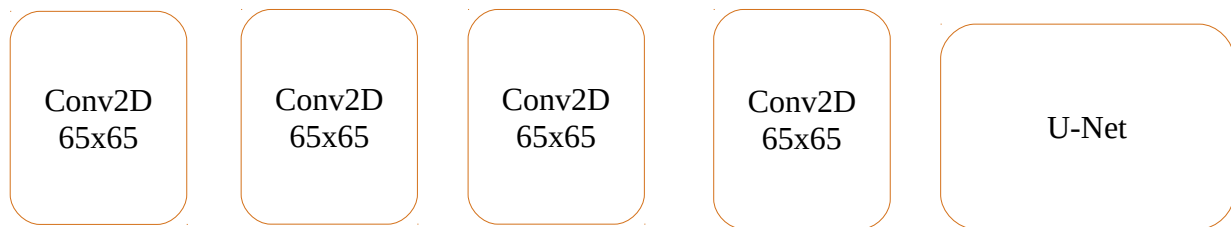


U-Net with hex representation

Input is right padded and up-sampled to reach dimension of 256x256 then convoluted to the dimension of 64x64 again after its output from pre-trained model.

Model has input channel of 3 and output of one
we concatenate the around the channel dimension of input.

We get output of CONV2D after the U-net to be of 16 channels



Name	Train Loss	Val Loss	Activation (Last layer)	Data type	Data size	Input dimension	Padding	# Epoch	LR	Comments
Simple	0.855	0.859	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	Still there is room for improvement Wasn't implemented correctly
	1.89902	1.89695	Softmax	Hex	Small	112x112	Right	6	1e-3	
	1.91967	1.90	Softmax	Hex	Small	112x112	Center	6	1e-3	
Four layer	0.0746	0.07503	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	Started to over-fit
	1.89902 4	1.89695	Softmax	Hex	Small	112x112	Center	6	1e-3	
	1.9233	1.92001 1	Softmax	Hex	Small	112x112	Center	6	1e-3	
Normali ze	0.08799	0.08835	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	Could be trained for more epochs
fl_max_ norm	0.08693	0.08731	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	
	1.90763	1.90757	Softmax	Hex	Small	112x112	Right	6	1e-3	
	1..91300	1.90990	Softmax	Hex	Small	112x112	Center	6	1e-3	
fl_max	.088807	.08920	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	Started to overfit
Combine d	.077447	.077867	Sigmoid	Bin	Small	112x112	Right	6 3	1e-2 1e-4	Takes more time
Al in All	.070321	.07069	Sigmoid	Bin	Small	112x112	Right	6 3	1e-2 1e-4	Best result
f_l_1_N orm_first	.08886	.089217 4	Sigmoid	Bin	Small	112x112	Right	6 3	1e-3 1e-4	Over-fit fastly
ResNet	.060458	.060137	Sigmoid	Bin	Small	112x112	Right	9 9 2 6	1e-2 1e-2 1e-1/2 1e-2	This phase trains only the last customized layers Unfreeze the whole model This will cause a jump in loss The model again started to decrease a cuda Actually resnet looks promising but didn't take chance to be trained more carefully with more data
U-Net Pre- trained	.000778	2.765e-5	Sigmoid	Bin	Medium	256x256	Right	1	1e-4	Best Result Takes Long time
	1.90944	1.90867	Softmax	Hex	Small	64x64	Right	1 2	1e-3/2 1e-4	The loss barely enhance
U-Net Scratch	0.21991 8	0.05173	Sigmoid	Bin	Medium	256x256	Right	1	1e-4	The loss was decreasing

Note: Last Layer activation implies the loss function used

U-net achieves the best results

We need to use a meter to compare between Cross entropy and binary cross entropy.

For consistency models need to be trained on the same amount of data.

Resnet could have a try specially because it's much faster than U-Net

U-Net achieves barely the same loss of much simpler architectures when dealing with hex data.